



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Βάσεις Δεδομένων

Αναφορά Εξαμηνιαίας Εργασίας

Διδάσκοντες:

Α. Τσουμάκος

Β. Στεφανίδης

Μ. Κόνιαρης

Παπαναστασίου Κωνσταντίνος Αριθμός Μητρώου: 03123067

Πράπα Άννα Αριθμός Μητρώου: 03123015

Προυσαλίδης Μιχαήλ Αριθμός Μητρώου: 03123002

Ακαδημαϊκό Έτος : 2025-2026

Περιεχόμενα

A	Σχεδιασμός και Υλοποίηση Βάσης Δεδομένων	3
A.1	ER διάγραμμα	3
A.2	Σχεσιακό διάγραμμα	4
A.2.1	Τύποι Δεδομένων	5
A.2.2	Περιορισμοί (Constraints)	5
A.2.3	Functions - Procedures - Triggers	10
A.2.4	Ευρετήρια (Indexes)	13
A.3	Views	14
A.4	Data	14
B	Queries	16
Γ	Εφαρμογή (User Interface)	28
Γ.1	Γενικά	28
Γ.2	Σελίδες	28
Γ.2.1	Αρχική σελίδα	28
Γ.2.2	Queries	29
Γ.2.3	Procedures	29
Γ.2.4	Admin page	32

Σχεδιασμός και Υλοποίηση Βάσης Δεδομένων

A.1 ER διάγραμμα

Παραθέτουμε κατωτέρω μερικές βασικές σχεδιαστικές επιλογές που κάναμε στο ER diagram και εν συνεχεία στο Relational Diagram. Καταρχάς, επειδή τόσο το προσωπικό του νοσοκομείου όσο και οι ασθενείς και οι συγγενείς τους είχαν κοινά στοιχεία μεταξύ τους, ορίσαμε ένα entity Human, από το οποίο κληρονομούσαν attributes όλες οι κατηγορίες ανθρώπων. Για τη διάκριση όλων των ανθρώπων χρησιμοποιήσαμε τον εγγενώς μοναδικό για κάθε άνθρωπο ΑΜΚΑ.

Ένας Human μπορεί να έχει πολλαπλά email και διαθέτει composite attribute του ονόματος. Ορίσαμε το phone ως οντότητα για να περιλαμβάνει επιπλέον πληροφορία τύπου τηλεφώνου

Από τον άνθρωπο, με βαθμιαία εξειδίκευση (specialization) δημιουργούνται τα ακόλουθα entities: ασθενής, συγγενής και προσωπικό, το οποίο εν συνεχεία διαχωρίζεται σε γιατρό, νοσηλευτή και διοικητικό προσωπικό. Το προσωπικό σχετίζεται με κάποια βάρδια (shift), entity το οποίο είναι weak καθώς χρειάζεται το department για να ταυτοποιηθεί.

Κάθε γιατρός μπορεί να έχει έναν επιβλέποντα ιατρό και συνδέεται μέσω ρόλων με την supervised_by. Το προσωπικό του νοσοκομείου ανήκει σε ένα ή περισσότερα τμήματα, τα οποία στεγάζονται σε συγκεκριμένους ορόφους κτηρίων.

Κάθε τμήμα έχει βάρδιες, οι οποίες καλύπτονται από το προσωπικό ενώ αντίστοιχα σχετίζεται με το floor, bed (weak entities για όμοιους λογους).

Οι ασθενείς εισάγονται στο νοσοκομείο και αρχικά υπόκεινται σε διαλογή. Το ER διάγραμμα εξασφαλίζει ότι κάθε ασθενής του νοσοκομείου έχει περάσει από διαλογή. Συγκεκριμένα, το entity triage καθορίζεται μονοσήμαντα από το ΑΜΚΑ του ασθενή και από την ακριβή χρονική στιγμή άφιξής του στο νοσοκομείο (primary key). Σε περίπτωση που κάποιος ασθενής χρειαστεί νοσηλεία, η νοσηλεία του καθορίζεται από τις ίδιες ακριβώς παραμέτρους, το ΑΜΚΑ του ασθενή και τη χρονική στιγμή άφιξής του στο νοσοκομείο. Έτσι, επιτρέπεται στον ίδιο ασθενή να νοσηλευτεί περισσότερες από μία φορές.

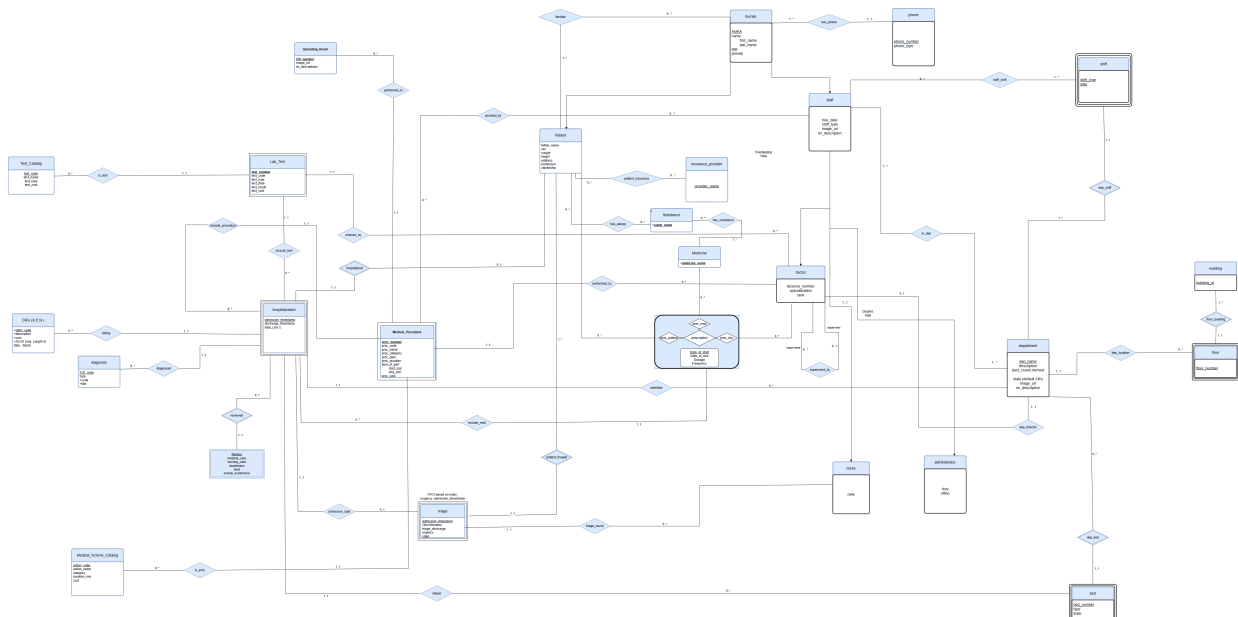
Κάθε νοσηλεία συνοδεύεται από μία διάγνωση εισόδου και μία διάγνωση εξόδου, οι οποίες ενδεχομένως να ταυτίζονται. Στο πλαίσιο μίας νοσηλείας, ένας ασθενής μπορεί να υποβληθεί σε εργαστηριακές εξετάσεις, ιατρικές πράξεις και φαρμακευτικές αγωγές.

Καθώς κάθε εξέταση, ιατρική πράξη και αγωγή θα πρέπει να ακολουθεί ευρωπαϊκά και εθνικά πρότυπα, ορίζουμε τα entities Test_catalog, Medical_Actions_Catalog, Substance και Medicine, τα οποία θα περιέχουν στη συνέχεια στη βάση μας όλες τις υπάρχουσες εργαστηριακές εξετάσεις, ιατρικές πράξεις και φάρμακα αντίστοιχα. Τα entities

Lab_Test, Medical_Procedure και Prescription αντιστοιχούν στις εξετάσεις, τις ιατρικές πράξεις και τις συνταγογραφήσεις αντίστοιχα που έχουν πραγματοποιηθεί στο νοσοκομείο και οι οποίες θεωρούμε ότι μπορούν να γίνουν μόνο στο πλαίσιο νοσηλείας. Μάλιστα κάθε εξέταση, ιατρική πράξη και συνταγογράφηση συνδέεται υποχρεωτικά με μία μόνο νοσηλεία. Όσον αφορά τις εξετάσεις, θεωρούμε ότι στη βάση δεδομένων του νοσοκομείου κάθε εξέταση χαρακτηρίζεται από έναν μοναδικό αύξοντα αριθμό. Το ίδιο ισχύει και για τις ιατρικές πράξεις και τις συνταγογραφήσεις. Επισημαίνουμε ακόμη ότι για τον σχηματισμό του entity prescription έχουμε χρησιμοποιήσει aggregation και έχουμε ορίσει ως primary key την τετράδα (ιατρός που χορηγεί, ασθενής που λαμβάνει, φάρμακο, ημερομηνία έναρξης).

Τέλος, μία νοσηλεία μπορεί να συνοδεύεται από μία μοναδική αξιολόγηση. Η αξιολόγηση είναι ένα weak entity που καθορίζεται πλήρως από το primary key της νοσηλείας στην οποία αναφέρεται. Κατωτέρω φαίνεται το ER diagram της βάσης μας.

Αναλύθηκαν τα πιο αξιολόγα entities του σχήματος και τα υπόλοιπα μπορούν να μελετηθούν παρακάτω, ενώ όλα υπόκεινται στους περιορισμούς πληθικότητας και συμμετοχής που ορίζει η εκφώνηση.

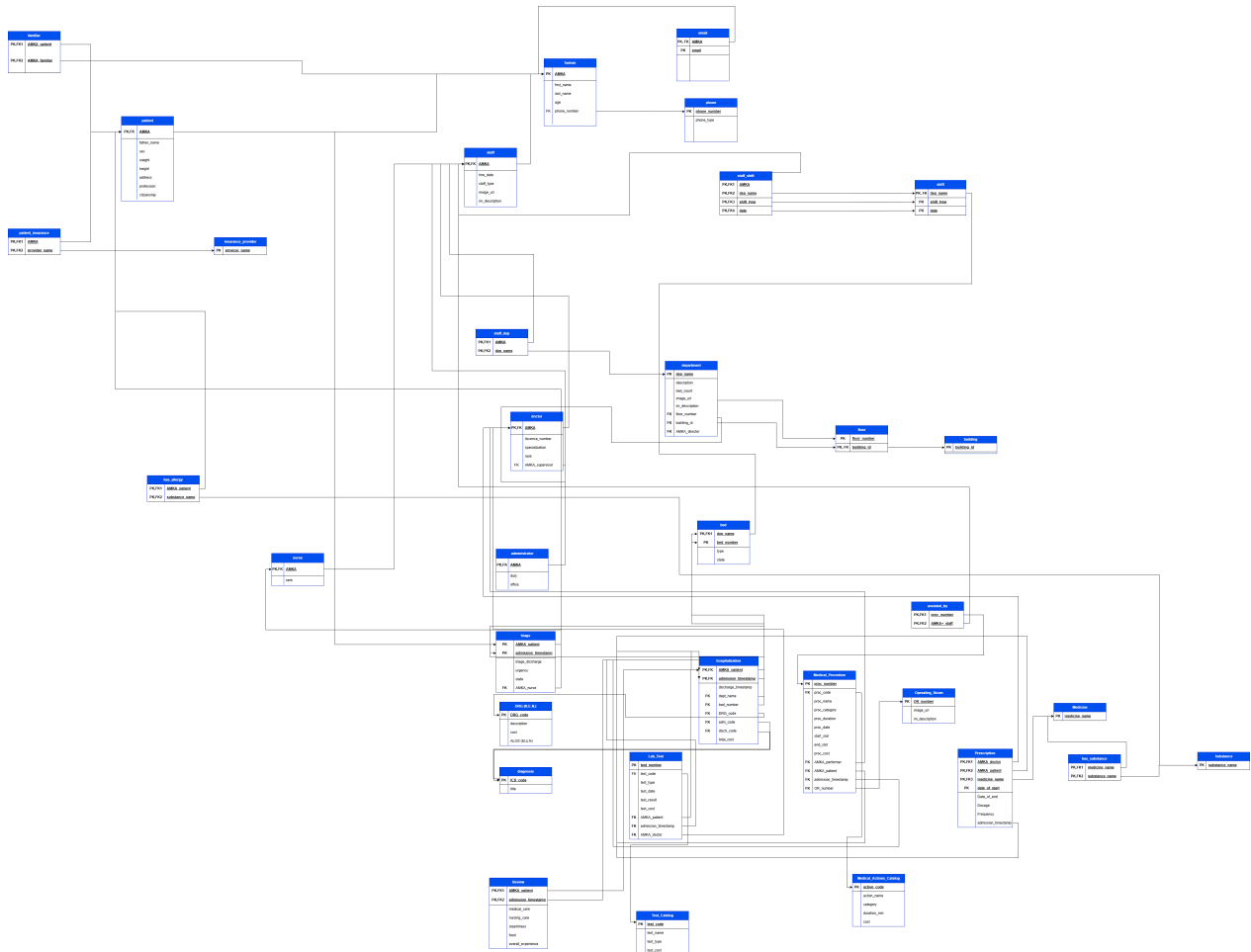


A.2 Σχεσιακό διάγραμμα

Βασιζόμενοι στο ER diagram, σχηματίζουμε το Relational diagram, εφαρμόζοντας τους γνωστούς κανόνες μετατροπής. Τόσο τα entities όσο και τα relationships μετατρέπονται σε relations. Όταν έχουμε multivalued attribute, δημιουργούμε ξεχωριστό αυτοτελές relation που συνδέεται με το αρχικό relation με ένα foreign key. Όταν δύο entities συνδέονται με relationship 1:N, ενσωματώνουμε τη σχέση ως attribute στο relation της πλευράς N το primary key του relation στην πλευρά N, επισημαίνοντας και την αντίστοιχη αναφορά foreign key. Όταν έχουμε σχέση N:N, δημιουργούμε σχέση με που περιέχει ως foreign keys τα primary keys των entities που συνδέονται. Ενδεικτικά αναφέρουμε ότι το multivalued attribute email στο relational diagram έχει αποσπαστεί ως ξεχωριστό entity, ενώ, επειδή κάθε bed ανήκει σε ένα department, έχουμε ενσωματώσει τη relationship dep_bed στο relation bed, προσθέτοντας στο bed ως attribute το dep_name με την αναφορά foreign key.

Όσον αφορά την εξειδίκευση, επιλέχθηκε η υλοποίηση με την αποθήκευση μόνο του PK των υπερκλάσεων στις υπο-

κλάσεις, σχεδιαστική επιλογή που αποφεύγει το redundancy ως προς τα δεδομένα και προσφέρει ευελιξία, ταχύτητα με την μερική αναζήτηση πληροφοριών για μια πραγματική οντότητα. Παραθέτουμε στην επόμενη σελίδα το relational diagram:



A.2.1 Τύποι Δεδομένων

Το πεδίο AMKA που αποτελεί primary key για πολλές οντότητες ορίζεται ως BIGINT προκειμένου να χωρά τα 11 ψηφία του. Όταν θέλουμε ακριβείς χρονικές στιγμές χρησιμοποιούμε τύπο δεδομένων TIMESTAMP, όταν θέλουμε ημερομηνίες χρησιμοποιούμε DATE. Για τα υπόλοιπα δεδομένα, εν γένει χρησιμοποιούμε ως τύπο το VARCHAR με μέγεθος ανάλογα με το αναμενόμενο μέγεθος για τα δεδομένα μας.

A.2.2 Περιορισμοί (Constraints)

Στον ορισμό της βάσης μας εισάγουμε τους ακόλουθους περιορισμούς

- CHECK (phone_type IN ('mobile','landline')): ο τύπος τηλεφώνου είναι είτε κινητό είτε σταθερό
- FOREIGN KEY (phone_number) REFERENCES phone(phone_number) ON DELETE RESTRICT ON UPDATE CASCADE: ο αριθμός τηλεφώνου ενός ανθρώπου θα πρέπει να υπάρχει στο table phone. Όπως και σε όλα τα επόμενα foreign key constraints, σε περίπτωση μεταβολής των στοιχείων η αλλαγή μεταδίδεται, ενώ η διαγραφή εμποδίζεται

- FOREIGN KEY (AMKA) REFERENCES human(AMKA) ON DELETE CASCADE ON UPDATE CASCADE: το AMKA στο table email θα πρέπει να αντιστοιχεί σε υπαρκτό AMKA στο table Human
- CHECK (staff_type IN ('doctor','administrator','nurse')): ένα μέλος του προσωπικού μπορεί να είναι είτε γιατρός είτε νοσηλευτής είτε μέλος του διοικητικού προσωπικού
- FOREIGN KEY (AMKA) REFERENCES human(AMKA) ON DELETE CASCADE ON UPDATE CASCADE: το AMKA ενός μέλους του προσωπικού θα πρέπει να αναφέρεται σε υπαρκτό AMKA στο table Human
- FOREIGN KEY (AMKA) REFERENCES staff (AMKA) ON DELETE CASCADE ON UPDATE CASCADE: το AMKA ενός νοσηλευτή θα πρέπει να αναφέρεται σε υπαρκτό AMKA στο table Human
- CHECK (rank IN ('assistant nurse','nurse','supervisor')): η βαθμίδα ενός νοσηλευτή είναι είτε Βοηθός Νοσηλευτή είτε Νοσηλευτής είτε Προϊστάμενος
- CONSTRAINT fk_AMKA_nurse FOREIGN KEY (AMKA_nurse) REFERENCES nurse (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE: το AMKA του νοσηλευτή διαλογής αναφέρεται υποχρεωτικά σε υπαρκτό νοσηλευτή
- CONSTRAINT urgency_values CHECK (urgency >= 1 AND urgency <= 5): το επίπεδο επείγοντος ενός περιστατικού ορίζεται ως ακέραια τιμή από το 1 έως το 5
- CONSTRAINT state_values CHECK (state IN ('active', 'inactive')): ένα περιστατικό triage μπορεί να είναι είτε ενεργό είτε ανενεργό, αν έχει παρέλθει η ημερομηνία εξυπηρέτησής του
- CONSTRAINT fk_patient_human FOREIGN KEY (AMKA) REFERENCES human (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE
- CONSTRAINT fk_patient_triage FOREIGN KEY (AMKA) REFERENCES triage (AMKA_patient) ON DELETE RESTRICT ON UPDATE CASCADE: το AMKA του ασθενή θα πρέπει να αντιστοιχεί σε υπαρκτό AMKA στο table triage, δεδομένου ότι κάθε ασθενής περνά υποχρεωτικά από το στάδιο του τριάζ.
- FOREIGN KEY (AMKA_patient) REFERENCES patient (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE: ελέγχουμε αν ένας συγγενής είναι συγγενής υπαρκτού ασθενούς
- FOREIGN KEY (AMKA_familiar) REFERENCES human (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE: ο συγγενής θα πρέπει προηγουμένως να έχει δηλωθεί στο table Human
- FOREIGN KEY (AMKA) REFERENCES patient (AMKA) ON DELETE CASCADE ON UPDATE CASCADE: ελέγχουμε αν η ασφάλεια ενός ασθενούς αναφέρεται σε υπαρκτό ασθενή
- FOREIGN KEY (provider_name) REFERENCES insurance_provider (provider_name) ON DELETE RESTRICT ON UPDATE CASCADE: ελέγχουμε αν το όνομα του ασφαλιστικού φορέα υπάρχει στο table με τους ασφαλιστικούς φορείς
- FOREIGN KEY (AMKA) REFERENCES staff (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE: ελέγχουμε αν το AMKA του γιατρού αντιστοιχεί σε υπαρκτό AMKA μέλους του προσωπικού
- FOREIGN KEY (AMKA_supervisor) REFERENCES doctor (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE: ελέγχουμε αν το AMKA του επόπτη αντιστοιχεί σε AMKA ήδη εισηγμένου στη βάση γιατρού
- CHECK (rank IN ('trainee','consultant B', 'consultant A', 'director')): η βαθμίδα ενός γιατρού μπορεί να είναι είτε ειδικευόμενος είτε επιμελητής B είτε επιμελητή A είτε διευθυντής

- CONSTRAINT trainee_supervisor CHECK (NOT(rank = 'trainee' AND AMKA_supervisor IS NULL)): όταν ένας γιατρός είναι ειδικευόμενος, θα πρέπει υποχρεωτικά να έχει επόπτη
- FOREIGN KEY (AMKA) REFERENCES staff (AMKA) ON DELETE CASCADE ON UPDATE CASCADE: το AMKA ενός μέλους του διοικητικού προσωπικού θα πρέπει να αντιστοιχεί σε AMKA μέλους του προσωπικού
- FOREIGN KEY (building_id) REFERENCES building (building_id) ON DELETE RESTRICT ON UPDATE CASCADE: το κτήριο στο οποίο βρίσκεται ένας όροφος θα πρέπει να βρίσκεται στο table building
- FOREIGN KEY (floor_number, building_id) REFERENCES floor (floor_number, building_id) ON DELETE RESTRICT ON UPDATE CASCADE:
- FOREIGN KEY (floor_number, building_id) REFERENCES floor (floor_number, building_id) ON DELETE RESTRICT ON UPDATE CASCADE: ο όροφος στον οποίο στεγάζεται ένα τμήμα του νοσοκομείου θα πρέπει να έχει δηλωθεί στο table floor
- FOREIGN KEY (AMKA_director) REFERENCES doctor (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE: το AMKA του διευθυντή ενός τμήματος του νοσοκομείου θα πρέπει να είναι AMKA ενός δηλωμένου γιατρού στο table doctor
- FOREIGN KEY (AMKA) REFERENCES staff (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE: ο πίνακας staff_dep θα πρέπει να περιλαμβάνει AMKA υπαρκτού προσωπικού
- FOREIGN KEY (dep_name) REFERENCES department (dep_name) ON DELETE RESTRICT ON UPDATE CASCADE: ομοίως θα πρέπει να περιλαμβάνει ονόματα υπαρκτών τμημάτων
- FOREIGN KEY (dep_name) REFERENCES department (dep_name) ON DELETE RESTRICT ON UPDATE CASCADE: το τμήμα όπου ανήκει μία κλίνη θα πρέπει να αντιστοιχεί σε υπαρκτό τμήμα του νοσοκομείου
- CHECK (type IN ('ICU','single-bed','multi-bed')): ο τύπος μίας κλίνης είναι είτε ΜΕΘ είτε μονόκλινο είτε πολύκλινο
- CHECK (state IN ('free','occupied','under-maintenance')): η κατάσταση μίας κλίνης είναι είτε ελεύθερη είτε δεσμευμένη είτε υπό συντήρηση
- FOREIGN KEY (dep_name) REFERENCES department (dep_name) ON DELETE RESTRICT ON UPDATE CASCADE: η βάρδια ενός τμήματος θα πρέπει να αναφέρεται σε τμήμα που έχει εισαχθεί στο table department
- CHECK (shift_type IN ('morning','noon','night')): υπάρχουν τρεις τύπου βαρδιών: πρωινή, απογευματινή και νυχτερινή
- FOREIGN KEY (dep_name, shift_type, date) REFERENCES shift (dep_name, date) ON DELETE RESTRICT ON UPDATE CASCADE: στο table staff_shift θα πρέπει η βάρδια σπου ανατίθεται σε ένα μέλος του προσωπικού να έχει δηλωθεί στο table shift
- FOREIGN KEY (AMKA) REFERENCES staff (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE: ομοίως θα πρέπει να έχει δηλωθεί το AMKA του αντίστοιχου μέλους του προσωπικού
- CONSTRAINT fk_allergy_patient FOREIGN KEY (AMKA_patient) REFERENCES patient (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE: το AMKA του ασθενούς στο table has_allergy θα πρέπει να υπάρχει στο table patient

- CONSTRAINT fk_allergy_substance FOREIGN KEY (substance_name) REFERENCES substance (substance_name) ON DELETE RESTRICT ON UPDATE CASCADE: ομοίως θα πρέπει να υπάρχει και το όνομα της αντίστοιχης ουσίας στο substance
- CONSTRAINT cost_gret_zero CHECK (cost >= 0): το κόστος της νοσηλείας θα πρέπει να είναι μη αρνητικό
- CONSTRAINT ALOS_gret_zero CHECK (ALOS >= 0): η μέση διάρκεια νοσηλείας θα πρέπει να είναι μη αρνητική
- CONSTRAINT fk_AMKA_adm FOREIGN KEY (AMKA_patient, admission_timestamp) REFERENCES triage (AMKA_patient, admission_timestamp) ON DELETE RESTRICT ON UPDATE CASCADE: κάθε νοσηλεία αναφέρεται στο μοναδικό περιστατικό τριάζ από το οποίο προέκυψε
- CONSTRAINT fk_dep_bed FOREIGN KEY (dep_name, bed_number) REFERENCES bed (dep_name, bed_number) ON DELETE RESTRICT ON UPDATE CASCADE: η κλίνη στην οποία νοσηλεύεται ο ασθενής θα πρέπει να έχει δηλωθεί στο table bed
- CONSTRAINT fk_DRG_code FOREIGN KEY (DRG_code) REFERENCES DRG (DRG_code) ON DELETE RESTRICT ON UPDATE CASCADE: ο κωδικός KEN της νοσηλείας θα πρέπει να έχει δηλωθεί στο συγκεντρωτικό table DRG
- CONSTRAINT fk_adm_code FOREIGN KEY (adm_code) REFERENCES diagnosis (ICD_code) ON DELETE RESTRICT ON UPDATE CASCADE: η διάγνωση εισόδου θα πρέπει να αντιστοιχεί σε διάγνωση που να υπάρχει στο συγκεντρωτικό table diagnosis
- CONSTRAINT fk_disch_code FOREIGN KEY (disch_code) REFERENCES diagnosis (ICD_code) ON DELETE RESTRICT ON UPDATE CASCADE: ομοίως και για τη διάγνωση εξόδου
- CONSTRAINT check_category CHECK (category IN ('Supportive (Anesthesia)', 'Surgical', 'Diagnostic', 'Therapeutic')): μία ιατρική πράξη μπορεί να είναι είτε υποστηρικτική είτε χειρουργική είτε διαγνωστική
- FOREIGN KEY (AMKA_performer) REFERENCES doctor (AMKA) ON UPDATE CASCADE ON DELETE RESTRICT: το AMKA του κύριου χειρουργού μίας επέμβασης πρέπει να αντιστοιχεί σε υπαρκτό AMKA γιατρού
- FOREIGN KEY (AMKA_patient, admission_timestamp) REFERENCES hospitalization(AMKA_patient, admission_timestamp) ON UPDATE CASCADE ON DELETE RESTRICT: κάθε ιατρική πράξη αντιστοιχεί σε μία υπαρκτή νοσηλεία
- FOREIGN KEY (OR_number) REFERENCES Operating_Room(OR_number) ON UPDATE CASCADE ON DELETE RESTRICT: κάθε ιατρική πράξη θα πρέπει να λαμβάνει χώρα σε χώρο που έχει δηλωθεί στο table operating_room
- FOREIGN KEY (proc_code) REFERENCES medical_actions_catalog (action_code) ON UPDATE CASCADE ON DELETE RESTRICT: ο κωδικός της ιατρικής πράξης θα πρέπει να υπάρχει στο συγκεντρωτικό table medical_actions_catalog
- CHECK (proc_category IN ('Supportive (Anesthesia)', 'Surgical', 'Diagnostic', 'Therapeutic')): μία ιατρική πράξη μπορεί να είναι είτε υποστηρικτική είτε χειρουργική είτε διαγνωστική
- CONSTRAINT timestamp_check CHECK (start_slot < end_slot): η στιγμή έναρξης της πράξης θα πρέπει να προηγείται της στιγμής λήξης της

- CONSTRAINT date_check CHECK (proc_date=DATE(start_slot)): η ημερομηνία της εξέτασης θα πρέπει να ταυτίζεται με την ημέρα της στιγμής έναρξης
- FOREIGN KEY (proc_number) REFERENCES Medical_Procedure(proc_number) ON UPDATE CASCADE ON DELETE CASCADE: η ιατρική πράξη στην οποία αναφέρεται το table assisted_by θα πρέπει να είναι μία υπαρκτή ιατρική πράξη
- FOREIGN KEY (AMKA_staff) REFERENCES staff(AMKA) ON UPDATE CASCADE ON DELETE RESTRICT: ομοίως το προσωπικό θα πρέπει να αντιστοιχεί σε υπαρκτά μέλη του προσωπικού
- CONSTRAINT check_test_type CHECK (test_type IN ('Imaging', 'Microbiology', 'Clinical')): το είδος μίας εξέτασης μπορεί να είναι είτε Απεικονιστική είτε Μικροβιολογική είτε Κλινική
- FOREIGN KEY (AMKA_patient, admission_timestamp) REFERENCES hospitalization(AMKA_patient, admission_timestamp) ON UPDATE CASCADE ON DELETE RESTRICT: η νοσηλεία στην οποία αναφέρεται μία εργαστηριακή εξέταση πρέπει να υπάρχει στο table Hospitalization
- FOREIGN KEY (AMKA_doctor) REFERENCES doctor(AMKA) ON UPDATE CASCADE ON DELETE RESTRICT: ο γιατρός που παρήγγειλε την εξέταση θα πρέπει να είναι υπαρκτός γιατρός
- FOREIGN KEY (test_code) REFERENCES Test_Catalog(test_code) ON UPDATE CASCADE ON DELETE RESTRICT: ο κωδικός της εξέτασης θα πρέπει να υπάρχει στον συγκεντρωτικό πίνακα Test_Catalog
- CONSTRAINT fk_medicine FOREIGN KEY (medicine_name) REFERENCES Medicine(medicine_name) ON UPDATE CASCADE ON DELETE RESTRICT: το όνομα του φαρμάκου στο οποίο αναφέρεται το table has_substance πρέπει να υπάρχει στο συγκεντρωτικό table Medicine
- CONSTRAINT fk_substance FOREIGN KEY (substance_name) REFERENCES substance(substance_name) ON UPDATE CASCADE ON DELETE RESTRICT: ομοίως η ουσία θα πρέπει να αναφέρεται στο συγκεντρωτικό table substance
- CONSTRAINT fk_prescription_doctor FOREIGN KEY (AMKA_doctor) REFERENCES doctor(AMKA) ON UPDATE CASCADE ON DELETE RESTRICT: ο γιατρός που χορηγεί μία αγωγή θα πρέπει να υπάρχει στο table doctor
- CONSTRAINT fk_prescription_hospitalization FOREIGN KEY (AMKA_patient, admission_timestamp) REFERENCES hospitalization(AMKA_patient, admission_timestamp) ON UPDATE CASCADE ON DELETE RESTRICT: η νοσηλεία στην οποία αναφέρεται η συνταγογράφηση θα πρέπει να υπάρχει στο table hospitalization
- CONSTRAINT fk_prescription_medicine FOREIGN KEY (medicine_name) REFERENCES Medicine(medicine_name) ON UPDATE CASCADE ON DELETE RESTRICT: το φάρμακο που συνταγογραφείται στον συγκεντρωτικό πίνακα Medicine
- CONSTRAINT date_check CHECK (date_of_start < date_of_end): θα πρέπει η ημερομηνία έναρξης της συνταγογράφησης να είναι προγενέστερη της ημερομηνίας λήξης της
- FOREIGN KEY (AMKA_patient, admission_timestamp) REFERENCES hospitalization(AMKA_patient, admission_timestamp) ON UPDATE CASCADE ON DELETE CASCADE: η νοσηλεία στην οποία αναφέρεται η αξιολόγηση θα πρέπει να έχει ήδη δηλωθεί στο table hospitalization

- CHECK (medical_care >= 1 AND medical_care <= 5): η αξιολόγηση γίνεται σε κλίμακα Likert επομένως οι τιμές της αξιολόγησης για την ποιότητα ιατρικής φροντίδας θα πρέπει να είναι ακέραιες τιμές από 1 έως και 5
- CHECK (nursing_care >= 1 AND nursing_care <= 5): ομοίως για τη νοσηλευτική φροντίδα
- CHECK (cleanliness >= 1 AND cleanliness <= 5): ομοίως για την ποιότητα της καθαριότητας
- CHECK (food >= 1 AND food <= 5): ομοίως για το φαγητό
- CHECK (overall_experience >= 1 AND overall_experience <= 5): ομοίως για τη συνολική εμπειρία

A.2.3 Functions - Procedures - Triggers

Κάθε function/procedure/trigger βρίσκεται εντός block που αρχίζει με ορισμό νέου delimiter '\$\$' και τελειώνει με την επαναφορά του προκαθορισμένου ';', για να αγνοούνται τα ';' και να δημιουργείται ολόκληρο το function/procedure/trigger.

Τα stored procedures (διαδικασίες) είναι blocks από SQL εντολές που αποθηκεύονται στη βάση και μπορούν να εκτελεστούν επαναλαμβανόμενα με στόχο την εφαρμογή της επιχειρηματικής λογικής.

Οι συναρτήσεις (functions) παίρνουν παραμέτρους εισόδου και μετά από υπολογισμούς, επιστρέφουν μια τιμή. Δεν καλούνται αυτόνομα, όπως τα procedures με CALL, αλλά μπορούν να ενσωματωθούν μέσα σε SQL ερωτήματα (π.χ. SELECT, WHERE).

Τα triggers είναι blocks κώδικα που εκτελούνται αυτόματα όταν συμβεί ένα συγκεκριμένο γεγονός. Χρησιμοποιούνται για την διατήρηση της ακεραιότητας των δεδομένων, πριν ή μετά από INSERT/UPDATE/DELETE.

- **FUNCTION** get_rank_value(rank_name VARCHAR(30)): Συνάρτηση που αντιστοιχίζει κάθε rank των ιατρών σε αριθμό 1-4. Χρησιμοποιείται σε επόμενα triggers.
- **TRIGGER** supervisor_check_insert BEFORE INSERT ON doctor: Ενεργοποιείται κάθε φορά που εισάγεται ιατρός. Σε περίπτωση που έχει επιβλέποντα, καλώντας την get_rank_value συγκρίνονται οι βαθμοί επιβλεπόμενου επιβλέποντα, κι αν ο δεύτερος δεν είναι αυστηρά μεγαλύτερος από τον πρώτο, αποτρέπεται η εισαγωγή.
- **TRIGGER** supervisor_check_update BEFORE UPDATE ON doctor: Ομοίως με παραπάνω για update.
- **TRIGGER** staff_dep_cardinality BEFORE INSERT ON staff_dep: Ενεργοποιείται κατά την εισαγωγή προσωπικού. Με βάσει το staff_type ελέγχει αν δεν είναι ιατρός, και δεν είναι, ελέγχει αν υπάρχει εγγραφή στο staff_dep (ανήκει ήδη σε ένα τμήμα) τότε αποτρέπει την εισαγωγή.
- **TRIGGER** staff_dep_cardinality_update BEFORE UPDATE ON staff_dep: Ομοίως με παραπάνω για update.
- **PROCEDURE** shift_staff_insert: Ορίζεται η διαδικασία για την εισαγωγή μιας βάρδιας, προκειμένου να γίνουν σεβαστά τα αντίστοιχα κριτήρια. Συγκεκριμένα, ορίσαμε ότι θα πρέπει να εισάγεται μια λίστα από AMKA με τα AMKA του προσωπικού και το αντίστοιχο αναγνωριστικό της βάρδιας (dep_name, shift_type, date). Τα AMKA αποθηκεύονται προσωρινά σε ένα TEMPORARY TABLE. Με JOIN με το table staff, καταμετρείται το doctor_count, nurse_count, administration_count και γίνονται οι αντίστοιχοι έλεγχοι. Αν πληρούνται, με JOIN των AMKA με το doctor καταμετρείται το trainee_count, senior_count και αν το πρώτο είναι >0 και το δεύτερο = 0 τότε απορρίπτεται η εισαγωγή. Αν πληρείται το κριτήριο, εκτελείται ένα INSERT query για κάθε AMKA. Η παραπάνω διαδικασία αναλαμβάνει τους ελέγχους, καθώς σε μορφή trigger θα έγειρε σφάλμα κατά την εισαγωγή μόνο ενός υπαλλήλου.

- **TRIGGER** shift_monthly_count BEFORE INSERT ON staff_shift: Ενεργοποιείται κατά την εισαγωγή υπαλλήλου σε βάρδια με την προηγούμενη διαδικασία. Συγκεκριμένα για την νέα εισαγωγή επιλέγονται σε CTE (nearby_shifts) οι βάρδιες του υπαλλήλου που υπάρχουν ήδη την ίδια, την προηγούμενη ή την επόμενη ημέρα. Έπειτα αθροίζεται το πλήθος των βαρδιών που είναι νύχτα την προηγούμενη ημέρα, αν η εισαγόμενη είναι πρωινή, των βαρδιών που είναι πρωί την επόμενη μέρα αν η εισαγόμενη είναι βραδινή και των βαρδιών που είναι πρωί η βράδι αν η εισαγόμενη είναι μεσημέρι. Αν το άθροισμα είναι μη μηδενικό εντοπίζεται βάρδια χωρίς διάλειμμα 8 ωρών και απορρίπτεται η εισαγωγή

Έπειτα σε CTE (nights) ενώνονται οι βάρδιες που είναι νύχτα και βρίσκονται έως και 3 μερες κοντά στην εισαγόμενη βάρδια. Έπειτα υπολογίζεται η αριθμηση γραμμής καθεμίας από αυτές και αυτός ο αριθμός αφαιρείται από κάθε ημερομηνία και τέλος ομαδοποιούνται αντίστοιχα με αυτή τη διαφορά. Αν υπάρχει group με πλήθος 4 τότε σημαίνει ότι η εισαγόμενη βάρδια προκαλεί 4 συνεχόμενες νυχτερινές βάρδιες και η εισαγωγή αποτρέπεται.

- **TRIGGER** shift_concurrency BEFORE INSERT ON staff_shift: εξασφαλίζει ότι κανένα μέλος του προσωπικού δε θα εργάζεται σε διαδοχικές βάρδιες ούτε θα βρεθεί σε πάνω από 3 συνεχόμενες νυχτερινές βάρδιες
- **TRIGGER** insert_beds: Το trigger φροντίζει, κατά την προσθήκη μίας κλίνης, να αυξηθεί κατά 1 το πλήθος των κλινών του αντίστοιχου τμήματος του νοσοκομείου, ώστε το attribute bed_count να παραμείνει έγκυρο.
- **TRIGGER** delete_beds: Το trigger φροντίζει, κατά τη διαγραφή μίας κλίνης να μειωθεί κατά 1 το πλήθος των κλινών του αντίστοιχου τμήματος του νοσοκομείου, ώστε το attribute bed_count να παραμείνει έγκυρο.
- **TRIGGER** update_beds: Το trigger μεριμνά ώστε, σε περίπτωση μεταφοράς μίας κλίνης από ένα τμήμα σε ένα άλλο, να μειωθεί το attribute bed_count κατά 1 στο αρχικό τμήμα και να αυξηθεί κατά 1 στο τελικό τμήμα.
- **PROCEDURE** calc_total_cost(IN my_AMKA_patient BIGINT, IN my_admission_timestamp TIMESTAMP, OUT total_cost_result DECIMAL(10,2)): Η διαδικασία υπολογίζει το κόστος νοσηλείας ενός ασθενούς βάσει της κοστολόγησης που παρέχει ο πίνακας των Κλειστών Ενοποιημένων Νοσηλίων (πηγή 4). Δέχεται ως παραμέτρους εισόδου (IN) το AMKA του ασθενούς και την ώρα εισαγωγής του στο νοσοκομείο, και βρίσκει από τον πίνακα hospitalization τον κώδικα K.E.N. και την ώρα αποχώρησης από το νοσοκομείο. Βάσει του κωδικού K.E.N. βρίσκει από τον πίνακα DRG, στον οποίο έχουν περαστεί τα ισχύοντα δεδομένα, το βασικό κόστος και την αναμενόμενη διάρκεια νοσηλείας (M.Δ.N.). Μέσω της DATEDIFF() υπολογίζεται η διάρκεια νοσηλείας. Αν η διάρκεια νοσηλείας δεν ξεπερνά την M.Δ.N., το τελικό κόστος είναι το βασικό, ενώ αν η διάρκεια νοσηλείας ξεπερνά τη M.Δ.N., προστίθεται καθορισμένη χρέωση 50 € ανά ημέρα. Η διαδικασία επιστρέφει (OUT) το τελικό κόστος.
- **PROCEDURE** release_procedure(IN my_AMKA_patient BIGINT, IN my_admission_timestamp TIMESTAMP): Αποτελεί τη διαδικασία εξιτηρίου του ασθενούς. Όσο ο ασθενής νοσηλεύεται, η ώρα αποχώρησής του είναι άγνωστο και συμβολίζεται με NULL. Με παραμέτρους εισόδου το AMKA του ασθενούς και την ώρα εισαγωγής, η ώρα αποχώρησης από το νοσοκομείο ορίζεται ως η ώρα εκτέλεσης της διαδικασίας. Αν δεν γίνει αλλαγή στον πίνακα, είτε ο συγκεκριμένος ασθενής έχει ήδη πάρει εξιτήριο, είτε ο ασθενής δεν έχει εισαχθεί.
- **TRIGGER** auto_calc_cost BEFORE UPDATE ON hospitalization: Όσο ο ασθενής νοσηλεύεται, το τελικό κόστος νοσηλείας είναι άγνωστο και συμβολίζεται με NULL. Το trigger φροντίζει για την ενημέρωση του πεδίου κατά την ενημέρωση του πίνακα hospitalization από τη διαδικασία εξιτηρίου.
- **TRIGGER** TRIGGER check_if_bed_free BEFORE INSERT ON hospitalization: Το trigger ελέγχει αν είναι ελεύθερο το κρεβάτι που δηλώνει ο χρήστης κατά την εισαγωγή του ασθενούς, πριν την εισαγωγή της εγγραφής στον πίνακα hospitalization. Αν το κρεβάτι είναι 'occupied', προκύπτει σφάλμα.

- **TRIGGER** bed_occupied_after_admission AFTER INSERT ON hospitalization: To trigger μεριμνά για την μεταβολή της κατάστασης του κρεβατιού που δηλώθηκε κατά την εισαγωγή του ασθενούς, από 'free' σε 'occupied', μετά την εισαγωγή της εγγραφής στον πίνακα hospitalization.
- **TRIGGER** bed_free_after_discharge AFTER UPDATE ON hospitalization: To trigger μεριμνά για την μεταβολή της κατάστασης του κρεβατιού από 'occupied' σε 'free', μετά την ενημέρωση του πίνακα hospitalization με την ώρα αποχώρησης (διαδικασία release_procedure).
- **PROCEDURE** copy_medical_procedure_fields: κατά την εισαγωγή μίας ιατρικής πράξης στο table Medical_Procedure, ο χρήστης συμπληρώνει τον κωδικό της εξέτασης και το σύστημα μόνο του αντιγράφει το όνομα, την κατηγορία και το κόστος της εξέτασης από το συγκεντρωτικό table Medical_Actions_Catalog, για εξασφάλιση της ακεραιότητας των δεδομένων.
- **PROCEDURE** copy_lab_test_fields: ομοίως, κατά την εισαγωγή μίας εργαστηριακής εξέτασης αντιγράφουμε τον τύπο και το κόστος της εξέτασης από τον συγκεντρωτικό πίνακα Test_Catalog
- **TRIGGER** operation_room_insert BEFORE INSERT ON Medical_Procedure: To trigger ελέγχει, κατά την εισαγωγή ενός χειρουργείου/αίθουσας επέμβασης ως χώρου επέμβασης εάν υπάρχει ήδη προγραμματισμένη άλλη επέμβαση στον ίδιο χώρο και σε ώρα που επικαλύπτεται χρονικά με τη νέα επέμβαση.
- **TRIGGER** operation_room_update BEFORE UPDATE ON Medical_Procedure: To trigger ελέγχει, κατά την ανανέωση ενός χειρουργείου/αίθουσας επέμβασης ως χώρου επέμβασης εάν υπάρχει ήδη προγραμματισμένη άλλη επέμβαση στον ίδιο χώρο και σε ώρα που επικαλύπτεται χρονικά με τη νέα επέμβαση.
- **TRIGGER** doctor_operation_insert BEFORE INSERT ON Medical_Procedure: To trigger ελέγχει κατά την εισαγωγή του κύριου χειρουργού μίας επέμβασης αν ο ίδιος γιατρός συμμετέχει είτε ως κύριος χειρουργός είτε ως βοηθητικό προσωπικό σε άλλη επέμβαση σε επικαλυπτόμενο χρονικό διάστημα με την υπό εξέταση επέμβαση.
- **TRIGGER** doctor_operation_update BEFORE UPDATE ON Medical_Procedure: To trigger ελέγχει κατά την ανανέωση του κύριου χειρουργού μίας επέμβασης αν ο ίδιος γιατρός συμμετέχει είτε ως κύριος χειρουργός είτε ως βοηθητικό προσωπικό σε άλλη επέμβαση σε επικαλυπτόμενο χρονικό διάστημα με την υπό εξέταση επέμβαση.
- **TRIGGER** assistant_operation_insert BEFORE INSERT ON assisted_by: To trigger ελέγχει κατά την εισαγωγή ενός μέλους του προσωπικού ως βοηθητικού προσωπικού σε μία επέμβαση αν το ίδιο μέλος συμμετέχει είτε ως κύριος χειρουργός είτε ως βοηθητικό προσωπικό σε άλλη επέμβαση σε επικαλυπτόμενο χρονικό διάστημα με την υπό εξέταση επέμβαση.
- **TRIGGER** assistant_operation_update BEFORE UPDATE ON assisted_by: To trigger ελέγχει κατά την ανανέωση ενός μέλους του προσωπικού ως βοηθητικού προσωπικού σε μία επέμβαση αν το ίδιο μέλος συμμετέχει είτε ως κύριος χειρουργός είτε ως βοηθητικό προσωπικό σε άλλη επέμβαση σε επικαλυπτόμενο χρονικό διάστημα με την υπό εξέταση επέμβαση.
- **TRIGGER** allergy_patient_insert BEFORE INSERT ON prescription: To trigger μεριμνά ώστε, κατά την εισαγωγή μίας συνταγογράφησης, ένας ασθενής να μη λάβει φάρμακο που έχει δραστική ουσία στην οποία είναι αλλεργικός.
- **TRIGGER** allergy_patient_update BEFORE UPDATE ON prescription: To trigger μεριμνά ώστε, κατά την ανανέωση μίας συνταγογράφησης, ένας ασθενής να μη λάβει φάρμακο που έχει δραστική ουσία στην οποία είναι αλλεργικός.
- **TRIGGER** review_discharged BEFORE INSERT ON review: To trigger φροντίζει η εισαγωγή μίας αξιολόγησης μίας νοσηλείας να μπορεί να γίνει μόνο μετά το τέλος της νοσηλείας

A.2.4 Ευρετήρια (Indexes)

Η βάση δημιουργεί αυτόματα indexes βάσει των Primary Keys των πινάκων. Πίνακες, όπως οι phone και human, που έχουν ένα μόνο PK, έχουν τα αντίστοιχα indexes (phone_number) και (AMKA). Πίνακες με σύνθετο PK, όπως οι triage με PRIMARY KEY (AMKA_patient, admission_timestamp) και bed με PRIMARY KEY (dep_name, bed_number), έχουν αυτόματο index είτε για το σύνολο του σύνθετου κλειδιού είτε μόνο για την πρώτη στήλη του κλειδιού. Για παράδειγμα η αναζήτηση για το admission_timestamp του triage δε γίνεται βέλτιστα από τα υπάρχοντα αυτόματα indexes και απαιτεί νέο index. Η βάση φτιάχνει, επίσης, αυτόματα indexes για στήλες που αναφέρονται σε FK περιορισμούς. Στόχος είναι να αποφεύγουμε τα full table scans, φτιάχνοντας indexes σε πεδία που μπαίνουν συχνά σε WHERE, ORDER BY και GROUP BY, ακολουθώντας το Left-Prefix Rule, που ορίζει ότι ένα σύνθετο index σε τρία πεδία μπορεί να χρησιμοποιηθεί για αναζήτηση στο πρώτο πεδίο, στο πρώτο και στο δεύτερο ή στο σύνολο, και αναδεικνύει τη σημασία της σειράς των πεδίων στη δήλωση ενός index. Παρατίθενται τα indexes ανά πίνακα:

1. staff_shift

- **CREATE INDEX** idx_staff_shift_concurrency ON staff_shift (date, shift_type);
Το index βελτιστοποιεί το trigger shift_concurrency το οποίο υπολογίζει το πλήθος των «κοντινών» βαρδιών με χρήση των συναρτήσεων SUM(), DATE_ADD() και DATE_SUB(). Ακόμη, εξυπηρετεί το ερώτημα 12 που φιλτράρει βάσει date και shift_type, μέσω WHERE και GROUP_BY, καθώς και το ερώτημα 8, όπου το date αποτελεί κλειδί της ένωσης των πινάκων staff και shift. Το ερώτημα 2 αξιοποιεί το index με τη χρήση του date σε WHERE, για το φιλτράρισμα αποτελεσμάτων που αφορούν το τρέχον έτος.
- **CREATE INDEX** idx_staff_shift_streak ON staff_shift (AMKA, shift_type, date);
Το index βελτιστοποιεί το CTE nights του trigger shift_concurrency και το trigger shift_monthly_count. Αμφότερα φιλτράρουν βάσει των AMKA και date, μέσω WHERE.

2. Medical_Procedure:

CREATE INDEX idx_medical_procedure_rooms ON Medical_Procedure (start_slot, end_slot, OR_number);
Βελτιστοποιεί τα triggers operation_room_insert και operation_room_update που διασφαλίζουν πως δεν υπάρχουν επικαλύψεις επεμβάσεων στην ίδια αίθουσα χειρουργείου. Οι έλεγχοι γίνονται μέσω WHERE και σύζευξης συνθήκης για το OR_number με συνθήκη για τα start_slot και end_slot. Βελτιστοποιεί επίσης τα triggers doctor_operation_insert και doctor_operation_update που ελέγχει ότι δεν υπάρχει ιατρός που εκτελεί δύο επεμβάσεις ταυτόχρονα, μέσω WHERE και συνθήκης για τα start_slot και end_slot.

3. assisted_by

CREATE INDEX idx_assisted_by_staff ON assisted_by (AMKA_staff, proc_number);
Το PK του πίνακα είναι το (proc_number, AMKA_staff). Το trigger doctor_operation_insert αναζητά μέσω WHERE αν ο κύριος χειρουργός της επέμβασης είναι δηλωμένος σε άλλη επέμβαση, χρησιμοποιώντας μόνο το AMKA_staff. Λόγω του Left-Prefix Rule, το αυτόματο index που έχει δημιουργηθεί για το σύνθετο PK δεν αρκεί.

4. has_substance

CREATE INDEX idx_has_substance_reverse ON has_substance (substance_name, medicine_name);
Το PK του πίνακα είναι το (medicine_name, substance_name). Τα triggers allergy_patient_insert και allergy_patient_update εκτελούν JOIN στους πίνακες has_allergy και has_substance, με ένωση στο substance_name, οπότε δε μπορούν να αξιοποιήσουν το αυτόματο index που δημιουργείται στο PK. Το ερώτημα 7 συνδέει πλέον τους πίνακες με βάση το substance_name. Το index παρέχει το σωστό αριστερό πρόθεμα (substance_name), επιτρέποντας στη βάση να κάνει Point Lookup (type: ref). Λειτουργεί ως Covering Index, εκτελώντας το ερώτημα στη μνήμη RAM.

5. prescription

CREATE INDEX idx_prescription_allergy_check ON prescription (AMKA_patient, medicine_name); Διευκολύνει τα triggers allergy_patient_insert και allergy_patient_update που αναζητούν αλλεργίες ανά ασθενή. Οι έλεγχοι γίνονται μέσω WHERE και σύζευξης συνθήκης για το AMKA_patient και το medicine_name.

6. hospitalization

- CREATE INDEX idx_hospitalization_discharge ON hospitalization (discharge_timestamp, dep_name, DRG_code);

Το ερώτημα 1 ομαδοποιεί βάσει discharge_timestamp, dep_name, DRG_code με GROUP BY. Το ερώτημα 9 φιλτράρει βάσει discharge_timestamp, μετατρέποντας ένα full scan σε range scan

- CREATE INDEX idx_hospitalization_group_cost ON hospitalization (AMKA_patient, dep_name, total_cost);

Το index αποτελεί composite covering index για το ερώτημα 3. Η βάση ούτε δεσμεύει προσωρινή μνήμη ούτε τρέχει αλγόριθμο ταξινόμησης, και απαντάει το ερώτημα από τη μνήμη RAM.

7. doctor

CREATE INDEX idx_doctor_specialization ON doctor (specialization, AMKA);

Το ερώτημα 2 φιλτράρει με το specialization για να βρει τους Καρδιολόγους. Το index λειτουργεί ως covering index.

8. human

CREATE INDEX idx_human_age ON human (age, AMKA);

Το ερώτημα 5 φιλτράρει με το specialization για να βρει τους ιατρούς κάτω των 35 ετών. Το index επιτρέπει στον optimizer να απομονώσει τους ανθρώπους με age < 35 μέσω Index Range Scan.

9. patient_insurance

CREATE INDEX idx_AMKA_provider_name ON patient_insurance (AMKA, provider_name);

Το index επιτρέπει στο ερώτημα 1 να κάνει άμεσο Key Lookup για το AMKA και να φέρει και το provider_name.

A.3 Views

- **VIEW** count_doctor_age_procedures: Η λειτουργία του εξηγήθηκε αναλυτικά στο κομμάτι του ερωτήματος 5.
- **VIEW** triage_queue: Γνωρίζουμε ότι κατά την άφιξη στο νοσοκομείο ο ασθενής περνά από διαλογή και καταγράφεται το επίπεδο επείγοντος του περιστατικού, με 1 το πιο επείγον και 5 το λιγότερο επείγον. Το view παρουσιάζει το AMKA, την ώρα εισαγωγής στα επείγοντα και το επίπεδο επείγοντος σε μορφή ουράς εξυπηρέτησης, έχοντας στην κορυφή τον ασθενή που πρέπει να νοσηλευτεί πρώτος. Με LEFT JOIN του πίνακα triage και του hospitalization, οι ασθενείς που έχουν περάσει το στάδιο διαλογής χωρίς να νοσηλευθούν έχουν στη στήλη του h.AMKA_patient τιμή NULL. Με WHERE κρατάμε αυτές τις γραμμές και τις ταξινομούμε βάσει επιπέδου επείγοντος σε αύξουσα σειρά, και ύστερα βάσει ώρα εισαγωγής στα επείγοντα. Έτσι, μεταξύ ασθενών που έχουν τον ίδιο βαθμό επείγοντος, πρώτος θα εξυπηρετηθεί εκείνος που έφτασε πρώτος.

A.4 Data

Αρχικά, έγινε επεξεργασία των δοσμένων δεδομένων ώστε να κρατήσουμε μόνο τα στοιχεία που χρειαζόμασταν για τη βάση. Επισημαίνουμε ειδικά για την περίπτωση των φαρμάκων και των δραστικών ουσιών ότι, στην περίπτωση φαρμάκων με περισσότερες από μία δραστικές ουσίες που χωρίζονταν στο αρχείο με «|», τις διαχωρίσαμε και δημιουργήσαμε

στους αντίστοιχους πίνακες περισσότερες από μία εγγραφές. Ως προς τα υπόλοιπα δεδομένα, η επιλογή δεδομένων έγινε με τρόπο ώστε αφενός μεν να ικανοποιούνται οι primary key και foreign key constraints, τα λοιπά constraints που θεωρήσαμε κατά τον ορισμό των tables και τα triggers που έχουμε ορίσει, αφετέρου δε να ικανοποιούνται και ορισμένοι βασικοί λογικοί περιορισμοί (π.χ. $0 \leq age \leq 150$), *.tables*, *sql/load.dql.*, *chcp65001*.

Queries

Q1.

Συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος, με ανάλυση ανά ΚΕΝ κωδικό και κατανομή νοσηλειών ανά ασφαλιστικό φορέα.

```
SELECT
    COALESCE(CAST(summary.RawYear AS CHAR), 'All Years') AS Year,
    COALESCE(summary.dep_name, 'All Departments') AS dep_name,
    COALESCE(summary.DRG_code, 'All DRGs') AS DRG_code,
    COALESCE(summary.provider_name, 'All Providers') AS provider_name,
    summary.Expected_Cost AS "Expected Cost",
    summary.Additional_Charge AS "Additional Charge",
    summary.Income AS Income
FROM (
    SELECT
        YEAR(h.discharge_timestamp) AS RawYear,
        h.dep_name,
        h.DRG_code,
        i.provider_name,
        SUM(d.cost) AS Expected_Cost,
        SUM(h.total_cost - d.cost) AS Additional_Charge,
        SUM(h.total_cost) AS Income
    FROM hospitalization h
    JOIN patient_insurance i ON h.AMKA_patient = i.AMKA
    JOIN DRG d ON d.DRG_code = h.DRG_code
    GROUP BY YEAR(h.discharge_timestamp), h.dep_name, h.DRG_code, i.provider_name
    WITH ROLLUP
) AS summary;
```

Αρχικά εμφανίζει τα επιμέρους έσοδα και μόλις παρουσιαστούν όλα τα έσοδα κάθε υποκατηγορίας εμφανίζει και τα συνολικά έσοδα της ανώτερης κατηγορίας

Q2.

Για ειδικότητα Καρδιολόγου όλοι οι γιατροί που ανήκουν σε αυτήν, με ένδειξη αν είχαν εφημερία το τρέχον έτος και πόσες επεμβάσεις εκτέλεσαν ως κύριοι χειρουργοί.

```
SELECT
```



```

d.AMKA AS AMKA,
d.specialization AS specialization,
CASE
    WHEN EXISTS (
        SELECT AMKA FROM staff_shift sh
        WHERE d.AMKA = sh.AMKA
        AND sh.date >= '2026-01-01'
        AND sh.date <= CURRENT_DATE
    ) THEN 'yes' ELSE 'no'
END AS had_night_shift_this_year,
COUNT(mp.proc_number) AS count_medical_procedures
FROM doctor d
LEFT JOIN medical_procedure mp ON d.AMKA = mp.AMKA_performer
WHERE d.specialization = 'Καρδιολογία'
GROUP BY d.AMKA, d.specialization;

```

Με το EXISTS το ερώτημα εντοπίζει τα AMKA των ιατρών που έχουν καταγεγραμμένη στον πίνακα staff_shift βάρδια το τρέχον έτος. Η ένδειξη βάρδιας παρουσιάζεται με 'yes'/'no', μέσω CASE. Με LEFT JOIN στους πίνακες doctor και medical_procedure και χρήση της COUNT() μετράμε το πλήθος των επεμβάσεων που έχει κάνει κάθε ιατρός. Τα αποτελέσματα φιλτράρονται βάσει ειδικότητας και ομαδοποιούνται κατά AMKA και ειδικότητα.

Q3.

Ασθενείς που έχουν νοσηλευτεί περισσότερες από 3 φορές στο ίδιο τμήμα, με το συνολικό κόστος νοσηλείας τους.

```

SELECT
    AMKA_patient,
    total_cost
FROM hospitalization
WHERE
    (AMKA_patient, dep_name) IN
    (SELECT
        AMKA_patient,
        dep_name
    FROM hospitalization
    GROUP BY AMKA_patient, dep_name
    HAVING COUNT(*) > 3
    ORDER BY AMKA_patient, dep_name);

```

Ο πίνακας των νοσηλειών οργανώνεται σε groups βάσει ασθενή και τμήματος, και από αυτά τα groups διαλέγονται όσα έχουν αριθμό εγγραφών > 3. Έπειτα από τον πίνακα των νοσηλειών, όσα ζεύγη (AMKA_patient, dep_name) ανήκουν στο προηγούμενο αποτέλεσμα εικονίζονται με το συνολικό τους κόστος.

Q4.

Για γιατρό 11015032143 μέσος όρος αξιολογήσεων των ασθενών του και η Συνολική εντύπωση νοσηλείας.

```

SELECT AVG(r.medical_care) as avg_medical_care, AVG(r.overall_experience)
AS avg_overall_experience
FROM ((

```

```

SELECT DISTINCT AMKA_patient, admission_timestamp
FROM prescription
WHERE AMKA_doctor = 11015032143) AS d
JOIN review AS r
  USING (AMKA_patient, admission_timestamp) );

```

Αρχικά επιλέγονται τα μοναδικά ζεύγη (*AMKA_patient, admission_timestamp*) από τις συνταγογραφήσεις του συγκεκριμένου ιατρού, καθώς καθένα από αυτά χαρακτηρίζει μια νοσηλία κι ένα review. Αρκεί ένα JOIN με τις κριτικές και AVG() aggregation για καθένα από τα ζητούμενα πεδία. Έγιναν PROJECT και SELECT το νωρίτερο, για την καθοδήγηση του query optimizer.

Για κάλυψη όλων των περιπτώσεων, έτρεξαν 3 παραλλαγές του query, μία χωρίς FORCE INDEX, μία με FORCE INDEX(PRIMARY) και μια με FORCE INDEX (misleading_index) με σκοπό τον αποπροσανατολισμό του query optimizer.

Φροντίσαμε κάθε query να τρέχει σε ξεχωριστή βάση, ώστε να αποκλείσουμε το ενδεχόμενο query caching.

(α)

```

MariaDB [opt1]> EXPLAIN
-> SELECT AVG(r.medical_care) as avg_medical_care, AVG(r.overall_experience) AS avg_overall_experience
-> FROM ((
->   SELECT DISTINCT AMKA_patient, admission_timestamp      FROM prescription
->   WHERE AMKA_doctor = 11015032143) AS d
-> JOIN review AS r
->   USING (AMKA_patient, admission_timestamp) );

```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	<derived2>	ALL	NULL	NULL	NULL	NULL	44	
1	PRIMARY	r	eq_ref	PRIMARY	PRIMARY	12	d.AMKA_patient,d.admission_timestamp	1	
2	DERIVED	prescription	ref	PRIMARY	PRIMARY	8	const	44	Using where; Using temporary

3 rows in set (0.003 sec)

(β)

```

MariaDB [opt2]> EXPLAIN
-> SELECT AVG(r.medical_care) as avg_medical_care, AVG(r.overall_experience) AS avg_overall_experience
-> FROM ((
->   SELECT DISTINCT AMKA_patient, admission_timestamp      FROM prescription FORCE INDEX (PRIMARY)
->   WHERE AMKA_doctor = 11015032143) AS d
-> JOIN review AS r FORCE INDEX (PRIMARY)
->   USING (AMKA_patient, admission_timestamp) );

```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	<derived2>	ALL	NULL	NULL	NULL	NULL	44	
1	PRIMARY	r	eq_ref	PRIMARY	PRIMARY	12	d.AMKA_patient,d.admission_timestamp	1	
2	DERIVED	prescription	ref	PRIMARY	PRIMARY	8	const	44	Using where; Using temporary

3 rows in set (0.001 sec)

CREATE INDEX misleading_index1 ON prescription (frequency); CREATE INDEX misleading_index2 ON review (nursing_care);

```

MariaDB [opt3]> EXPLAIN
-> SELECT AVG(r.medical_care) as avg_medical_care, AVG(r.overall_experience) AS avg_overall_experience
-> FROM ((
->   SELECT DISTINCT AMKA_patient, admission_timestamp      FROM prescription FORCE INDEX (misleading_index1)
->   WHERE AMKA_doctor = 11015032143) AS d
-> JOIN review AS r FORCE INDEX (misleading_index2)
->   USING (AMKA_patient, admission_timestamp) );

```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	r	ALL	NULL	NULL	NULL	NULL	499	
1	PRIMARY	<derived2>	ref	key0	key0	12	opt3.r.AMKA_patient,opt3.r.admission_timestamp	1	
2	DERIVED	prescription	ALL	NULL	NULL	NULL	NULL	3200	Using where; Using temporary

3 rows in set (0.001 sec)

(γ)

Παρατηρείται ότι χωρίς force index, ο query optimizer διαλέγει το βέλτιστο θεωρητικά query plan, δηλαδή τη χρήση του index από το primary key, καθώς πανομοιότυπο αποτέλεσμα προκύπτει και στη δεύτερη περίπτωση. Το κόστος σε γραμμές που εξετάζονται είναι πολύ μικρό.

Στην τρίτη περίπτωση, τα misleading_indexes επειδή δεν σχετίζονται με το query, ο optimizer δεν τα αξιοποιεί και εκτελεί full table scan (ALL), όπως αναδεικνύει το κόστος σε γραμμές που εξετάζονται.

Ως προς τον πραγματικό χρόνο, τα 2 πρώτα queries (χωρίς explain) εκτελέστηκαν σε 0.004 sec ενώ το 3ο σε 0.012 sec (τριπλάσιος χρόνος). Αυτό είναι λογικό καθώς τα IO accesses σε περίπτωση full table scan είναι γραμμικά του πλήθους των block σε ένα table.

Στην πρώτη το eq_ref υποδεικνύει αποδοτικό JOIN καθώς υπάρχει μοναδική εγγραφή που συνδέεται σε κάθε join μεταξύ των 2 πινάκων. Στη δεύτερη περίπτωση ο query optimizer επιχείρησε να κατασκευάσει ένα κλειδί key0 για την πιο αποδοτική υλοποίηση του join.

(δ)

Συμπεραίνεται ότι ειδικά σε μικρού μεγέθους σχέσεις, ο query optimizer αξιοποιώντας τα primary keys είναι ικανός να επιλέξει ένα execution plan αρκετά αποδοτικό που υλοποιεί αποδοτικά searches και joins. Αντίθετα, αν επιβληθεί index το οποίο είναι άσχετο με το query, ο optimizer το αγνοεί και σπαταλά χρόνο τόσο στο plan όσο και σε αναποδοτικά searches και joins.

Αν θεωρηθεί αξιόπιστος ο χρόνος εκτέλεσης του explain, μπορούμε επίσης να συμπεράνουμε ότι το FORCE INDEX(PRIMARY) είναι το βέλτιστο, καθώς ο optimizer δεν σπαταλά χρόνο ώστε να το διαπιστώσει.

Q5.

Νέοι ιατροί (ηλικία < 35 ετών) που έχουν εκτελέσει τις περισσότερες χειρουργικές επεμβάσεις ως κύριοι χειρουργοί.

```
CREATE OR REPLACE VIEW count_doctor_age_procedures AS
SELECT
    d.AMKA AS AMKA,
    h.age AS age,
    COUNT(mp.proc_number) AS count_medical_procedures
FROM doctor d
INNER JOIN human h ON d.AMKA = h.AMKA
LEFT JOIN medical_procedure mp ON d.AMKA = mp.AMKA_performer
GROUP BY d.AMKA, h.age
HAVING age < 35;

SELECT
    AMKA,
    age,
    count_medical_procedures
FROM count_doctor_age_procedures
```

```

WHERE count_medical_procedures = (
    SELECT MAX(count_medical_procedures)
    FROM count_doctor_age_procedures
);

```

Το view count_doctor_age_procedures παρουσιάζει το AMKA του ιατρού, την ηλικία και το πλήθος επεμβάσεων που έχει εκτελέσει ο ιατρός ως κύριος χειρουργός, για όλους τους ιατρούς ηλικίας κάτω των 35 ετών. Την ηλικία του ιατρού παίρνουμε με INNER JOIN του doctor με τον human. Το πλήθος των επεμβάσεων παίρνουμε με LEFT JOIN, για να τυπωθούν μηδενικές τιμές σε ιατρούς που δεν έχουν κάνει επεμβάσεις. Από το view count_doctor_age_procedures που απομονώνει το υποσύνολο ιατρών που είναι ηλικίας κάτω των 35 ετών παίρνουμε τις γραμμές που έχουν στο πλήθος επεμβάσεων το μέγιστο που εμφανίζεται στον πίνακα.

Q6.

Για ασθενή 11015000070 το ιστορικό νοσηλειών του, οι αντίστοιχες διαγνώσεις (ICD-10), το συνολικό κόστος ανά νοσηλεία και ο μέσος όρος αξιολόγησής του.

```

SELECT *
FROM (
    SELECT
        h.admission_timestamp,
        h.adm_code,
        h.disch_code,
        h.total_cost
    FROM hospitalization AS h
    WHERE h.AMKA_patient = 11015000070
) AS h
NATURAL JOIN (
    SELECT
        r.admission_timestamp,
        (r.medical_care + r.nursing_care + r.cleanliness + r.food + r.overall_experience) / 5
    AS avg_rating
    FROM review AS r
    WHERE r.AMKA_patient = 11015000070
) AS r;

```

Το αποτέλεσμα προκύπτει ως JOIN των εγγραφών των νοσηλειών και των αξιολογήσεων που αφορούν τον συγκεκριμένο ασθενή. Να σημειωθεί ότι έγινε column aggregation στις στήλες των αξιολογήσεων. Έγιναν PROJECT και SELECT το νωρίτερο, για την καθοδήγηση του query optimizer.

(α)

```
MariaDB [opt4]> EXPLAIN SELECT *
-> FROM
-> (
->   SELECT
->     h.admission_timestamp,
->     h.adm_code,
->     h.disch_code,
->     h.total_cost
->   FROM hospitalization AS h
->   WHERE h.AMKA_patient = 11015000070
-> ) AS h
-> NATURAL JOIN
-> (
->   SELECT
->     r.admission_timestamp,
->     (
->       r.medical_care +
->       r.nursing_care +
->       r.cleanliness +
->       r.food +
->       r.overall_experience
->     ) / 5 AS avg_rating
->   FROM review AS r
->   WHERE r.AMKA_patient = 11015000070
-> ) AS r;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	r	ref	PRIMARY	PRIMARY	8	const	2	
1	SIMPLE	h	eq_ref	PRIMARY,idx_hospitalization_group_cost	PRIMARY	12	const,opt4.r.admission_timestamp	1	

2 rows in set (0.003 sec)

(β)

```
MariaDB [opt5]> EXPLAIN SELECT * FROM (
->   SELECT h.admission_timestamp, h.adm_code, h.disch_code, h.total_cost
->   FROM hospitalization AS h FORCE INDEX (PRIMARY)
->   WHERE h.AMKA_patient = 11015000070
-> ) AS h
-> NATURAL JOIN (
->   SELECT r.admission_timestamp, ( r.medical_care + r.nursing_care + r.cleanliness + r.food + r.overall_experience ) / 5 AS avg_rating
->   FROM review AS r FORCE INDEX (PRIMARY)
->   WHERE r.AMKA_patient = 11015000070
-> ) AS r;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	h	ref	PRIMARY	PRIMARY	8	const	1	
1	SIMPLE	r	eq_ref	PRIMARY	PRIMARY	12	const,opt5.h.admission_timestamp	1	

2 rows in set (0.005 sec)

CREATE INDEX misleading_index1 ON hospitalization (adm_code); CREATE INDEX misleading_index2 ON review (nursing_care);

```
MariaDB [opt6]> EXPLAIN SELECT * FROM (
->   SELECT h.admission_timestamp, h.adm_code, h.disch_code, h.total_cost
->   FROM hospitalization AS h FORCE INDEX (misleading_index1)
->   WHERE h.AMKA_patient = 11015000070
-> ) AS h
-> NATURAL JOIN (
->   SELECT r.admission_timestamp, ( r.medical_care + r.nursing_care + r.cleanliness + r.food + r.overall_experience ) / 5 AS avg_rating
->   FROM review AS r FORCE INDEX (misleading_index2)
->   WHERE r.AMKA_patient = 11015000070
-> ) AS r;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	r	ALL	NULL	NULL	NULL	NULL	499	Using where
1	SIMPLE	h	ALL	NULL	NULL	NULL	NULL	508	Using where; Using join buffer (flat, BNL join)

2 rows in set (0.000 sec)

(γ)

Παρατηρείται ότι χωρίς force index, ο query optimizer διαλέγει το βέλτιστο θεωρητικά query plan, δηλαδή τη χρήση του index από το primary key (ref), καθώς πανομοιότυπο αποτέλεσμα προκύπτει και στη δεύτερη περίπτωση. Το κόστος σε γραμμές που εξετάζονται είναι πολύ μικρό (είναι από τις πρώτες εγγραφές του πίνακα).

Στην τρίτη περίπτωση, τα misleading_indexes επειδή δεν σχετίζονται με το query, ο optimizer δεν τα αξιοποιεί και εκτελεί full table scan (ALL), όπως αναδεικνύει το κόστος σε γραμμές που εξετάζονται. Επειδή επιπλέον δεν βρίσκει αποτελεσματικό index, εκτελεί loop join (BNL).

Ως προς τον πραγματικό χρόνο, τα 2 πρώτα queries (χωρίς explain) εκτελέστηκαν σε 0.005 και 0.004 sec ενώ το 3ο σε 0.001 sec. Αυτό μπορεί να ερμηνευτεί από το γεγονός ότι η επίλυση του query plan μπορεί να εισάγει overhead, το οποίο παρακάμπτεται από το full table scan.

(δ)

Συμπεραίνεται ότι ειδικά σε μικρού μεγέθους σχέσεις, ο query optimizer αξιοποιώντας τα primary keys είναι ικανός να επιλέξει ένα execution plan αρκετά αποδοτικό που υλοποιεί αποδοτικά searches και joins. Αντίθετα, αν επιβληθεί index το οποίο είναι άσχετο με το query, ο optimizer το αγνοεί και σπαταλά χρόνο τόσο στο plan όσο και σε αναποδοτικά searches και joins (BNL). Ωστόσο σε πολύ μικρά tables το full table scan είναι η πιο αποδοτική αναζήτηση

Q7.

Για κάθε δραστική ουσία τον αριθμό ασθενών που έχουν δηλώσει αλλεργία και τον αριθμό φαρμάκων που την περιέχουν, ταξινομημένα κατά συνολικό αριθμό αλλεργικών ασθενών.

```
SELECT
    s.substance_name AS substance,
    COUNT(DISTINCT ha.AMKA_patient) AS count_allergic_patients,
    COUNT(DISTINCT hs.medicine_name) AS count_medicine
FROM substance s
LEFT JOIN has_allergy ha ON s.substance_name = ha.substance_name
LEFT JOIN has_substance hs ON s.substance_name = hs.substance_name
GROUP BY s.substance_name
ORDER BY count_allergic_patients DESC;
```

Αρχίζοντας από τον πίνακα substance γίνονται διαδοχικά LEFT JOIN με τους πίνακες has_allergy και has_substance. Για την αποφυγή διπλότυπων γραμμών AMKA και φαρμάκων λόγω πολλαπλών JOIN χρησιμοποιείται το DISTINCT. Τα αποτελέσματα ομαδοποιούνται κατά όνομα ουσίας και ταξινομούνται σε φθίνουσα σειρά βάσει του αριθμού αλλεργικών ασθενών.

Q8.

Το προσωπικό που δεν έχει προγραμματισμένη εφημερία σε συγκεκριμένη ημερομηνία (2026-01-01) και τμήμα (Καρδιολογία).

```
SELECT s.AMKA
```

```

FROM staff s
LEFT JOIN staff_shift sh ON s.AMKA = sh.AMKA
    AND sh.dep_name = 'Cardiology'
    AND sh.date = '2026-01-01'
WHERE sh.AMKA IS NULL;

```

Εκτελούμε LEFT JOIN του πίνακα staff με τον πίνακα staff_shift κρατώντας τις γραμμές που αφορούν βάρδιες στο τμήμα Καρδιολογίας εντός του τρέχοντος έτους. Τα AMKA που δεν έχουν βάρδια στο τμήμα Καρδιολογίας του τρέχον έτος εμφανίζουν NULL τιμές στην στήλη sh.AMKA και είναι τα επιθυμητά αποτελέσματα.

Q9.

Ποιοι ασθενείς νοσηλεύτηκαν τον ίδιο αριθμό ημερών σε διάστημα ενός έτους, με συνολική διάρκεια άνω των 15 ημερών.

```

WITH bounded_hospitalizations AS (
    SELECT
        AMKA_patient,
        DATEDIFF(
            LEAST(discharge_timestamp, '2025-12-31 23:59:59'),
            GREATEST(admission_timestamp, '2025-01-01 00:00:00')
        ) AS duration_days
    FROM hospitalization
    WHERE admission_timestamp <= '2025-12-31 23:59:59'
        AND (discharge_timestamp >= '2025-01-01 00:00:00' OR discharge_timestamp IS NULL)
)
SELECT
    AMKA_patient,
    duration_days
FROM bounded_hospitalizations
WHERE duration_days > 15
    AND duration_days IN (
        SELECT duration_days
        FROM bounded_hospitalizations
        GROUP BY duration_days
        HAVING COUNT(DISTINCT AMKA_patient) > 1
    )
ORDER BY duration_days DESC, AMKA_patient ASC;

```

Ο έλεγχος γίνεται για το έτος 2025 όμως μπορεί εύκολα να επεκταθεί για οποιοδήποτε έτος ή χρονικό διάστημα.

Q10.

Τα top-3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα στον ίδιο ασθενή κατά την ίδια νοσηλεία, ταξινομημένα κατά συχνότητα εμφάνισης.

```

SELECT
    s1.substance_name AS substance_1,
    s2.substance_name AS substance_2,
    COUNT(*) AS frequency

```

```

FROM prescription p1
JOIN has_substance s1 ON p1.medicine_name = s1.medicine_name
JOIN prescription p2 ON p1.AMKA_patient = p2.AMKA_patient
    AND p1.admission_timestamp = p2.admission_timestamp
JOIN has_substance s2 ON p2.medicine_name = s2.medicine_name
WHERE s1.substance_name < s2.substance_name
GROUP BY s1.substance_name, s2.substance_name
ORDER BY frequency DESC
LIMIT 3;

```

Q11.

Ιατροί που έχουν εκτελέσει τουλάχιστον 5 λιγότερες επεμβάσεις από τον ιατρό με τις περισσότερες επεμβάσεις στο τρέχον έτος.

```

WITH doctors_rank AS (
    SELECT
        d.AMKA,
        count(mp.proc_number) AS count_procedures,
        MAX(count(mp.proc_number)) OVER () AS max_procedures
    FROM doctor d
    LEFT JOIN medical_procedure mp
    ON
        d.AMKA = mp.AMKA_performer
    AND mp.proc_date >= '2026-01-01'
    AND mp.proc_date <= CURRENT_DATE
    GROUP BY d.AMKA
)
SELECT
    AMKA,
    count_procedures
FROM doctors_rank
WHERE count_procedures <= max_procedures - 5;

```

Στο CTE doctors_rank παρουσιάζουμε το AMKA του ιατρού, το πλήθος επεμβάσεων και στήλη με τον αριθμό μέγιστων επεμβάσεων, που υπολογίζεται με την window function MAX() OVER(). Με LEFT JOIN του doctor με τον medical_procedure στο AMKA του ιατρού κρατάμε τις γραμμές που αφορούν επεμβάσεις το τρέχον έτος και ομαδοποιούμε βάσει AMKA. Από το CTE απομονώνουμε τις στήλες του AMKA και του πλήθους επεμβάσεων ανά ιατρό και φιλτράρουμε βάσει πλήθους επεμβάσεων, κρατώντας τα AMKA που έχουν εκτελέσει τουλάχιστον 5 λιγότερες επεμβάσεις από το μέγιστο.

Q12.

Ο απαιτούμενος αριθμός προσωπικού ανά τμήμα και ανά βάρδια για εβδομάδα 2026-01-01/08, με ανάλυση ανά υποκλάση προσωπικού.

```

SELECT
    ss.dep_name,
    ss.date,

```



```

ss.shift_type,
s.staff_type,
CASE
    WHEN s.staff_type = 'doctor' THEN d.specialization
    WHEN s.staff_type = 'nurse' THEN n.rank
    WHEN s.staff_type = 'administrator' THEN a.duty
END AS subclass_detail,
COUNT(ss.AMKA) AS required_staff_count
FROM staff_shift ss
LEFT JOIN staff s ON ss.AMKA = s.AMKA
LEFT JOIN doctor d ON s.staff_type = 'doctor' AND ss.AMKA = d.AMKA
LEFT JOIN nurse n ON s.staff_type = 'nurse' AND ss.AMKA = n.AMKA
LEFT JOIN administrator a ON s.staff_type = 'administrator' AND ss.AMKA = a.AMKA
WHERE ss.date BETWEEN '2026-01-01' AND '2026-01-08'
GROUP BY ss.dep_name, ss.date, ss.shift_type, s.staff_type,
CASE
    WHEN s.staff_type = 'doctor' THEN d.specialization
    WHEN s.staff_type = 'nurse' THEN n.rank
    WHEN s.staff_type = 'administrator' THEN a.duty
END
ORDER BY ss.dep_name, ss.date, ss.shift_type;

```

Τυπώνουμε όλες τις καταχωρημένες βάρδιες του νοσοκομείου την πρώτη εβδομάδα του 2026. Τροποποιώντας τις ημερομηνίες μπορούμε να πάρουμε τις καταχωρημένες βάρδιες για οποιοδήποτε χρονικό διάστημα.

Q13.

Για κάθε ιατρό όλη την ιεραρχία εποπτείας του, από τον άμεσο επόπτη έως τον Διευθυντή, με ένδειξη του επιπέδου σε κάθε βαθμίδα.

```

WITH RECURSIVE chain AS (
    SELECT
        AMKA AS start_AMKA,
        AMKA AS current_AMKA,
        rank,
        AMKA_supervisor,
        0 AS level
    FROM doctor
    UNION ALL
    SELECT
        c.start_AMKA,
        d.AMKA,
        d.rank,
        d.AMKA_supervisor,
        c.level + 1
    FROM doctor d
    JOIN chain c ON d.AMKA = c.AMKA_supervisor
)
SELECT start_AMKA, rank, level, AMKA_supervisor
FROM chain

```

```
ORDER BY start_AMKA, level;
```

Για το συγκεκριμένο query απαιτήθηκε αναδρομικός ορισμός. Η βάση της αναδρομής ορίζεται να ξεκινά λαμβάνοντας αρχικό AMKA μαζί με τη βαθμίδα και το AMKA του επιβλέποντα. Το βήμα της αναδρομής γίνεται με ένωση της εγγραφής με την αντίστοιχη εγγραφή του πίνακα ιατρών με AMKA το AMKA του επιβλέποντα. Σε κάθε ένωση ο αριθμητής επιπέδου αυξάνεται κατά 1.

Q14.

Ποιες κατηγορίες ICD-10 διαγνώσεων είχαν τον ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη, με τουλάχιστον 5 περιστατικά ανά έτος.

```
WITH codes_per_year AS (  
    SELECT  
        LEFT(adm_code, 1) AS ICD_category,  
        YEAR(admission_timestamp) AS year,  
        COUNT(AMKA_patient) AS count  
    FROM hospitalization  
    GROUP BY LEFT(adm_code, 1), YEAR(admission_timestamp)  
    HAVING count >= 5  
)  
SELECT DISTINCT cpy1.ICD_category  
FROM codes_per_year cpy1  
INNER JOIN codes_per_year cpy2  
    ON cpy1.ICD_category = cpy2.ICD_category  
    AND cpy1.count = cpy2.count  
    AND cpy2.year = cpy1.year + 1;
```

Το CTE codes_per_year απομονώνει τον πρώτο χαρακτήρα του κωδικού διάγνωσης που συμβολίζει την κατηγορία διάγνωσης, εξάγει το έτος από την ημερομηνία εισαγωγής του ασθενούς στο νοσοκομείο και μετρά το πλήθος εισαγωγών που αντιστοιχούν σε μια κατηγορία. Ομαδοποιεί τα αποτελέσματα ανά κατηγορία και ανά έτος, οπότε παράγονται γραμμές της μορφής 'A – 2025 – 11' και 'A – 2026 – 11' και κρατάει τις γραμμές με πλήθος εισαγωγών μεγαλύτερο ή ίσο του 5.

Με SELF JOIN στο CTE κρατάμε τις γραμμές που αφορούν κατηγορίες με ίδιο αριθμό εισαγωγών και με διαφορά ενός έτους ('A – 2025 – 11 – A – 2026 – 11'). Με το DISTINCT παρουσιάζονται τα μοναδικά ονόματα κατηγοριών που πληρούν τις προϋποθέσεις.

Q15.

Κατανομή των περιστατικών triage ανά επίπεδο επείγοντος, με μέσο χρόνο αναμονής ανά επίπεδο, ποσοστό περιστατικών που οδήγησαν σε νοσηλεία και κατανομή παραπομπών ανά τμήμα.

```
SELECT *  
FROM (  
    SELECT  
        urgency,  
        COUNT(*) AS distribution,  
        AVG(TIMESTAMPDIFF(MINUTE, admission_timestamp, triage_discharge)) AS mean_wait,  
        AVG(state = 'inactive') * 100 AS hospitalization_percentage
```

```

        FROM triage
        GROUP BY urgency
    ) AS t1
JOIN (
    SELECT
        t.urgency,
        h.dep_name,
        COUNT(*) AS dep_distr
    FROM (
        SELECT * FROM triage WHERE state = 'inactive'
    ) AS t
    JOIN hospitalization AS h
        ON t.AMKA_patient = h.AMKA_patient
        AND t.admission_timestamp = h.admission_timestamp
    GROUP BY t.urgency, h.dep_name
) AS t2 ON t1.urgency = t2.urgency;

```

Αρχικά γίνεται grouping με βάση το επίπεδο επείγοντος. Σε κάθε group γίνεται aggregation COUNT(*) για την κατανομή, AVG() για διαφορά των χρόνων εισαγωγής και αποχώρησης από triage, AVG() για ποσοστό που προχώρησαν σε νοσηλεία (σχεδιαστικά, το attribute state δηλώνει αν μια διαλογή είναι ενεργή ή όχι δηλ. αν έχει προχωρήσει σε νοσηλεία). Αυτό το αποτέλεσμα γίνεται JOIN με το αντίστοιχο grouping με βάση επείγον και όνομα τμήματος όπου γίνεται aggregation COUNT() για την κατανομή. Στην κατασκευή των δεδομένων υποτέθηκε ότι η κατανομή των admissions είναι ίδια για κάθε επίπεδο επείγοντος.

Εφαρμογή (User Interface)

Γ.1 Γενικά

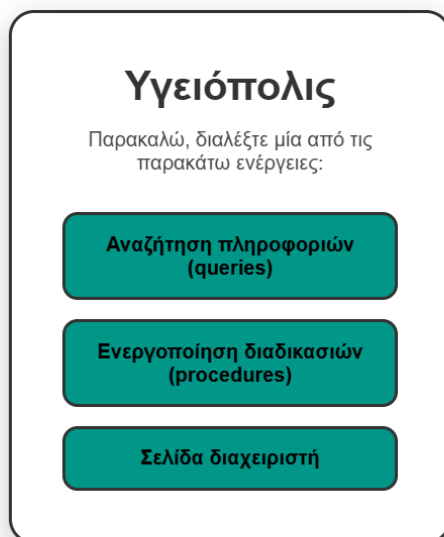
Η εφαρμογή υλοποιήθηκε σε γλώσσα PHP και βασίστηκε στα demo των διδασκόντων. Η χρήση εργαλείων τεχνητής νοημοσύνης περιορίστηκε μόνο στην αναζήτηση γρήγορου documentation της PHP, CSS και debugging.

Οι λειτουργίες που υποστηρίζει είναι (1) η κλήση των queries της εργασίας, (2) η επίδειξη των υλοποιημένων PROCEDURES της εργασίας και (3) η επίδειξη των tables της βάσης, των εικόνων και η δυνατότητα προσθαφαίρεσης δεδομένων.

Γ.2 Σελίδες

Γ.2.1 Αρχική σελίδα

Η αρχική σελίδα παραπέμπει τον χρήστη στις διαφορετικές σελίδες που υλοποιούν τις 3 κύριες λειτουργίες.



Γ.2.2 Queries

Αυτή η σελίδα παραπέμπει τον χρήστη σε κατάλογο με όλα τα διαθέσιμα queries της εργασίας και επιτρέπει την εκτέλεσή τους:



Database Queries

Q1: Συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος, με ανάλυση ανά KEN κωδικό ~

Run Query

Q1: Συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος, με ανάλυση ανά KEN κωδικό και κατανομή νοσηλείων ανά ασφαλιστικό φορέα.

Q2: Για ειδικότητα Καρδιολόγου όλοι οι γιατροί που ανήκουν σε αυτήν, με ένδειξη αν είχαν εφημερία το τρέχον έτος και πόσες επεμβάσεις εκτέλεσαν ως κύριοι χειρουργοί.

Q3: Ασθενείς που έχουν νοσηλευτεί περισσότερες από 3 φορές στο ίδιο τμήμα, με το συνολικό κόστος νοσηλείας τους.

Q4: Για γιατρό 11015032143 μέσος όρος αξιολογήσεων των ασθενών του και η Συνολική εντύπωση νοσηλείας

Q5: Νέοι ιατροί (ηλικία < 35 ετών) που έχουν εκτελέσει τις περισσότερες χειρουργικές επεμβάσεις ως κύριοι χειρουργοί.

Q6: Για ασθενή 11015000070 το ιστορικό νοσηλείων του, οι αντίστοιχες διαγνώσεις (ICD-10), το συνολικό κόστος ανά νοσηλεία και ο μέσος όρος αξιολόγησής του

Q7: Για κάθε δραστική ουσία τον αριθμό ασθενών που έχουν δηλώσει αλλεργία και τον αριθμό φαρμάκων που την περιέχουν, ταξινομημένα κατά συνολικό αριθμό αλλεργικ

Q8: Το προσωπικό που δεν έχει προγραμματισμένη εφημερία σε συγκεκριμένη ημερομηνία και τμήμα.

Q9: Ποιοι ασθενείς νοσηλεύτηκαν τον ίδιο αριθμό ημερών σε διάστημα ενός έτους, με συνολική διάρκεια άνω των 15 ημερών

Q10: Τα top-3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα στον ίδιο ασθενή κατά την ίδια νοσηλεία, ταξινομημένα κατά συχνότητα εμφάνισης

Q11: Ιατροί που έχουν εκτελέσει τουλάχιστον 5 λιγότερες επεμβάσεις από τον ιατρό με τις περισσότερες επεμβάσεις στο τρέχον έτος

Q12: Ο απαιτούμενος αριθμός προσωπικού ανά τμήμα και ανά βάρδια για εβδομάδα 2026-01-01/08, με ανάλυση ανά υποκλάση προσωπικού

Q13: Για κάθε ιατρό όλη την ιεραρχία επιπτώσεώς του, από τον άμεσο επόπτη έως τον Διευθυντή, με ένδειξη του επιπέδου σε κάθε βαθμίδα.

Q14: Ποιες κατηγορίες ICD-10 διαγνώσεων είχαν τον ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη, με τουλάχιστον 5 περιστατικά ανά έτος

Q15: Κατανομή των περιστατικών triage ανά επίπεδο επείγοντος, με μέσο χρόνο αναμονής ανά επίπεδο, ποσοστό περιστατικών που οδήγησαν σε νοσηλεία και κατανομή

Όταν επιλέγεται το query της φόρμας τίθεται η μεταβλητή \$selected της PHP με το αντίστοιχο αύξοντα αριθμό και με ένα refresh το frontend στέλνει το αντίστοιχο query από τον πίνακα queries στη βάση, και μόλις λάβει το αποτέλεσμα εικονίζει αρχικά τα fields και επαναληπτικά κάθε σειρά:



Database Queries

Q1: Συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος, με ανάλυση ανά KEN κωδικό ~

Run Query

Year	dep_name	DRG_code	provider_name	Expected Cost	Additional Charge	Income
2025	Cardiology	S01A	Ergo Ασφαλιστική	5400	5400.00	10800.00
2025	Cardiology	S01A	All Providers	5400	5400.00	10800.00
2025	Cardiology	S21X	Eurolife FFH	2320	2320.00	4640.00
2025	Cardiology	S21X	All Providers	2320	2320.00	4640.00
2025	Cardiology	B22A	Ergo Ασφαλιστική	5450	0.00	5450.00
2025	Cardiology	B22A	All Providers	5450	0.00	5450.00

Γ.2.3 Procedures

Σε αυτή τη σελίδα εκτελούνται οι διαδικασίες εισαγωγής βάρδιας, εξιτήριου ασθενή και αναζήτησης πληροφοριών για ιατρικές πράξεις και εργαστηριακές εξετάσεις βάσει κωδικού στους καταλόγους.

Εισαγωγή Βάρδιας

Επίδειξη:

11015008580,11015007338,11015012325,11015078227,11015091597,11015036274,11015006752,11015027883,11015069000,11015088774,11015072048,11015038810

Cardiology

noon

2026-08-09

AMKAs (comma-separated)
Department
Shift Type
Shift Date

Submit

Εξιτήριο ασθενή

11015057394, 2026-03-05 15:16:00 for demonstration

AMKA patient
Admission timestamp

Submit

Πληροφορίες για Ιατρική Πράξη

A400095 για επίδειξη

Procedure Code

Submit

Πληροφορίες για Εργαστηριακή Εξέταση

B300111 για επίδειξη

Lab Test Code

Submit

Τα αντίστοιχα πεδία δέχονται τα στοιχεία εισόδου και τα κάνουν sanitize όπου χρειάζεται (τα μετατρέπουν σε κατάλληλη μορφή για την κλήση CALL της διαδικασίας). Όταν λαμβάνουν αποτέλεσμα από τη βάση το εικονίζουν, αλλιώς εικονίζουν το σφάλμα. Παρακάτω παρατίθεται ενδεικτικά η εύρεση πληροφοριών για εργαστηριακή εξέταση:

Εισαγωγή Βάρδιας

Επίδειξη:

11015006688,11015087888,11015012326,11015079227,11015081687,11015086274,11015006752,11015027383,11015068000,11015083774,11015072046,11015038810

Cardiology

noon

2026-06-09

AMKAs (comma-separated)
Department
Shift Type
Shift Date

Submit

Εξιτήριο ασθενή

11015057394, 2026-03-05 15:16:00 for demonstration

AMKA patient
Admission timestamp

Submit

Πληροφορίες για Ιατρική Πράξη

A400095 για επίδειξη

Procedure Code

Submit

Πληροφορίες για Εργαστηριακή Εξέταση

B300111 για επίδειξη

Lab Test Code

Submit

Test Type: Microbiology
Test Cost: \$128.96

Τέλος υποστηρίζεται η δυνατότητα απεικόνισης η γραμμών από το view που αφορά τις FIFO triage QUEUES:

Submit

Πληροφορίες για Εργαστηριακή Εξέταση

B300111 για επίδειξη

Lab Test Code

Submit

Επόμενοι ασθενείς triage προς εξυπηρέτηση

6

Submit

AMKA_patient	admission_timestamp	urgency
12015000013	2025-05-29 05:33:00	1
12015000073	2025-06-09 03:21:00	1
12015000048	2025-07-20 21:54:00	1
12015000083	2025-08-09 21:59:00	1

Ενεργοποιήστε τα Windows

Γ.2.4 Admin page

Αυτή η σελίδα προσφέρει τη δυνατότητα απεικόνισης κάθε σχέσης στη βάση, την προσθήκη και τη διαγραφή εγγραφών.

🏠

Η σελίδα του διαχειριστή

Η εισαγωγή γίνεται με τη σειρά που εμφανίζονται οι σχέσεις για σεβασμό της αναφορικής ακεραιότητας

phone

Απεικόνιση σχέσης

Εισαγωγή στο phone

phone_number

phone_type

Insert Row

phone_number	phone_type	Actions
102-974-4788	landline	Delete
105-860-5421	mobile	Delete

Για την απεικόνιση, με όμοιο τρόπο υλοποιήθηκε drop down menu για τον ορισμό του \$selectedTable. Με βάσει αυτό, εικονίζονται τα forms για την εισαγωγή κάθε attribute του.

Η σελίδα του διαχειριστή

Η εισαγωγή γίνεται με τη σειρά που εμφανίζονται οι σχέσεις για σεβασμό της αναφορικής ακεραιότητας

phone

phone

human

email

staff

nurse

triage

patient

familiar

insurance_provider

patient_insurance

Απεικόνιση σχέσης

phone_number

phone_number

phone

phor

ow

Η διαφορά με τις άλλες σελίδες είναι ότι κατά την απεικόνιση του table προστίθεται ένα κουμπί διαγραφής. Καθώς έχουν αρχικοποιηθεί τα διαφορετικά primary keys των δεδομένων, η διαγραφή γίνεται βάσει αυτών. Τέλος, γίνεται κατάλληλο handling σε περίπτωση που τα δεδομένα περιέχουν url εικόνας.

🏠

Η σελίδα του διαχειριστή

Η εισαγωγή γίνεται με τη σειρά που εμφανίζονται οι σχέσεις για σεβασμό της αναφορικής ακεραιότητας

staff

Απεικόνιση σχέσης

Εισαγωγή στο staff

AMKA

hire_date

staff_type

image_url

im_description

AMKA

hire_date

staff_type

image_url

im_description

Insert Row

AMKA	hire_date	staff_type	Image	Actions
11015001275	2003-06-23	nurse		Delete

Ορόσηπος οδήγησης στην αποκατάσταση αράχωσης

Ενεργοποιήστε τα Windows

Σχήμα Γ.1: Staff

dep_name

dep_name

description

description

floor_number

floor_number

building_id

building_id

AMKA_director

AMKA_director

image_url

image_url

im_description

im_description

Insert Row

dep_name	description	floor_number	building_id	AMKA_director	Image	Actions
Cardiology	Χώρος πλήρως ανακαινισμένος με εργονομικό σχεδιασμό για ασθενείς.	1	1	11015026999	 Προηγμένη τεχνολογική υποστήριξη για σύνθετα ιατρικά περιστατικά.	Delete
Emergency	Τμήμα στελεχωμένο με κορυφαίους επαγγελματίες υγείας.	1	5	11015064821	 Προηγμένη τεχνολογική υποστήριξη για σύνθετα ιατρικά περιστατικά.	Delete
Gastroenterology	Σύγχρονος εξοπλισμός και	3	4	11015080947		Delete

Σχήμα Γ.2: Department

OR_number

OR_number

image_url

image_url

im_description

im_description

Insert Row

OR_number	Image	Actions
1		Delete

Σχήμα Γ.3: Operating room